



《AI大模型Ragent项目》——大模型没有记忆多轮对话怎么做到不失忆？

来自： 拿个offer-开源&项目实战

👤 马丁

2026年04月20日 22:04

开篇引言

上一篇画了 StreamChatPipeline 的全景地图，八个阶段从加载记忆到流式生成，一行 execute() 方法 20 行代码就是整个问答系统的骨架。你可能注意到了，阶段 1 只有一行调用：

```
loadMemory(ctx); // 阶段 1
```

看起来很轻，但这一行背后藏着记忆系统的完整设计——存储、加载、并行拉取、降级容错、摘要注入，全都压缩在这个方法调用里。来看一个具体场景。假设你在一家企业做内部知识库助手，有个员工连续问了三个关于 OA 系统的问题：

- 第 1 轮：OA 系统怎么提交加班申请？
- 第 2 轮：提交之后审批流程是怎样的？
- 第 3 轮：如果它超过三天没审批怎么办？

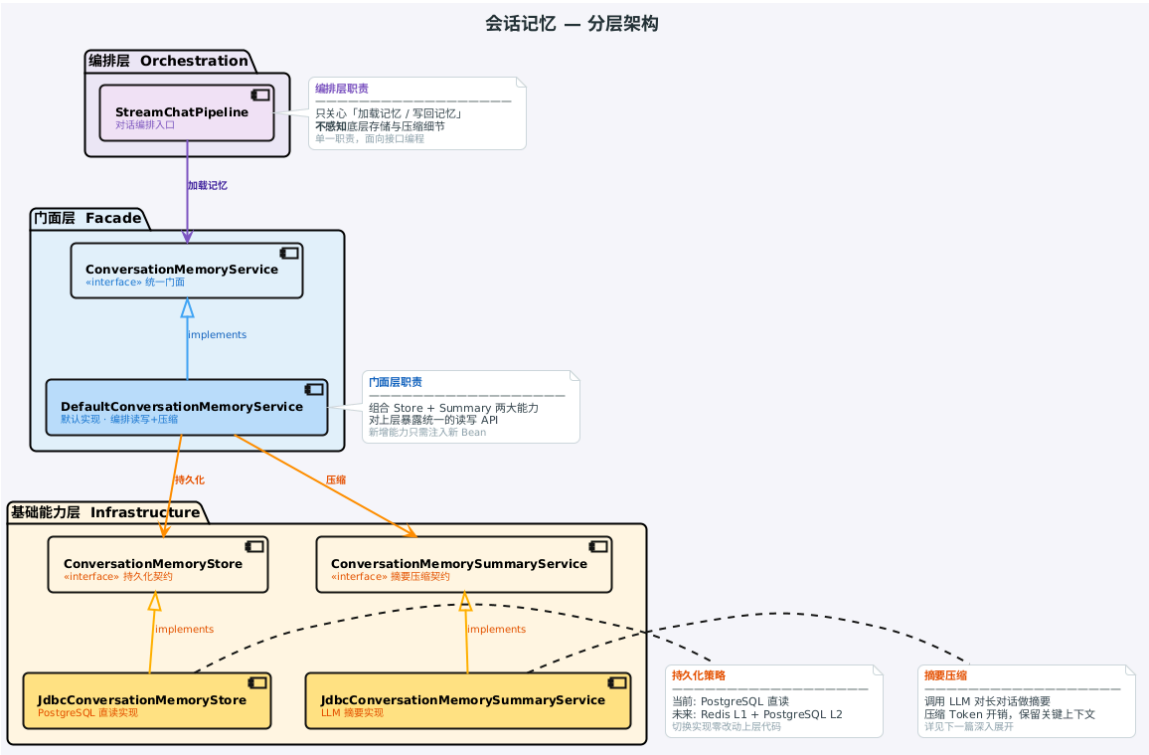
第 3 轮的“它”指的是加班申请，“三天没审批”承接了第 2 轮的审批流程。大模型的 API 每次请求都是独立的，它不记得之前聊了什么。如果你不把前两轮对话塞进去，模型看到的就只是一句“如果它超过三天没审批怎么办”——它不知道“它”是什么，也不知道审批是哪个审批。这就是记忆要解决的核心问题：**让每次独立的 API 调用看起来像一段连贯的对话。**

但记忆不是无脑把所有历史都塞进去就完事了。塞多了 Token 预算爆了，加载慢了用户体验差，存储方案选错了扩展性堵死。本篇聚焦记忆的存储与加载——记忆从哪来、怎么存、怎么加载、怎么装进消息数组。至于记忆太长了怎么办——摘要压缩策略，那是下一篇的事。

记忆系统的整体架构

1. 三层设计

Ragent 的记忆系统不是一个大类把什么都干了，而是拆成了三层，每层各管各的事：



用一句话概括每层的职责：

层级	核心角色	职责
编排层	StreamChatPipeline	只管调 loadMemory，不关心记忆怎么来的
门面层	ConversationMemoryService	统一暴露 load / append / loadAndAppend 三个方法，屏蔽底层差异

基础能力层	MemoryStore + SummaryService	一个管持久化读写，一个管摘要压缩，各自独立
-------	------------------------------	-----------------------

2. 为什么这样分层

你可能会觉得，就一个记忆功能，至于拆这么细吗？实际上这三层各自解决一个问题：

门面层屏蔽存储差异。 当前版本的持久化方案是 PostgreSQL 直读（MyBatis-Plus），接口上预留了 refreshCache 方法为未来的 Redis 缓存做准备，但当前实现是空操作。假设下个版本要加 Redis 做缓存，只需要新写一个 RedisConversationMemoryStore 实现 ConversationMemoryStore 接口，门面层和编排层一行代码不用动。

持久化和压缩独立演进。 换存储方案不影响压缩逻辑，换压缩策略也不影响存储。比如你想把摘要压缩从单次 LLM 调用改成增量式压缩，只需要替换 ConversationMemorySummaryService 的实现，持久化那边完全无感。

编排层只关心结果。 Pipeline 调 loadMemory 拿到一个 List<ChatMessage> 就够了，它不需要知道这个列表是从 PostgreSQL 查出来的还是从 Redis 拿的，也不需要知道里面有没有摘要。

消息的持久化：append 做了什么

用户发了一条消息，或者模型回了一句话，都要存下来。append 方法就是干这个事的。

1. 存储到 PostgreSQL

每条消息存在 t_message 表里，核心字段长这样：

字段	类型	说明
id	String（雪花 ID）	主键
conversation_id	String	会话 ID
user_id	String	用户 ID
role	String	user 或 assistant
content	String	消息内容
thinking_content	String	深度思考链（可空）
thinking_duration	Integer	思考耗时秒数（可空）
create_time	Date	自动填充
deleted	Integer	逻辑删除（0=有效，1=删除）

几个细节值得注意：

1.1 雪花 ID 一举两得

主键用 MyBatis-Plus 的雪花算法自动生成，雪花 ID 本身就是按时间递增的，所以它既是主键又是天然的时间排序字段。后面讲摘要压缩时，还会用它当水位线——这次摘要覆盖到哪条消息了，直接比较 ID 大小就行，不需要额外的时间戳字段。

1.2 USER 和 ASSISTANT 消息的处理不同

USER 消息会额外触发 conversationService.createOrUpdate，更新会话记录的 lastTime——这是给前端会话列表排序用的，最近聊过的会话排在最前面。ASSISTANT 消息则会保存 thinkingContent 和 thinkingDuration，这两个字段只在深度思考模式下有值。

2. 异步触发摘要压缩

append 方法除了存消息，还有一个附带效果——异步触发摘要压缩：

```
@Override
public String append(String conversationId, String userId, ChatMessage message) {
    String messageId = memoryStore.append(conversationId, userId, message);
    // 追加完消息后，异步触发摘要压缩（仅 ASSISTANT 消息触发）
    summaryService.compressIfNeeded(conversationId, userId, message);
    return messageId;
}
```

这里有两个设计考量：

只有 ASSISTANT 消息才触发压缩。一轮完整对话是 user + assistant 各一条消息，assistant 消息到了说明这一轮结束了，这时候去检查要不要压缩才有意义。如果 user 消息就触发，assistant 的回复还没来，压缩出来的摘要少了半轮对话。

压缩是异步的，不阻塞主流程。compressIfNeeded 内部会判断消息轮数有没有达到阈值，达到了才真正发起压缩。压缩过程涉及 LLM 调用，耗时不确定，放在异步线程里不会拖慢当前请求的响应速度。

压缩的具体实现——LLM 怎么生成摘要、水位线怎么更新、分布式锁怎么防并发，这些都是下一篇的内容，这里先知道 append ASSISTANT 消息后会异步触发一下就够了。

历史的加载：load 做了什么

存进去了，怎么拿出来？load 方法是记忆系统最核心的读取逻辑。

1. 并行拉取摘要和历史

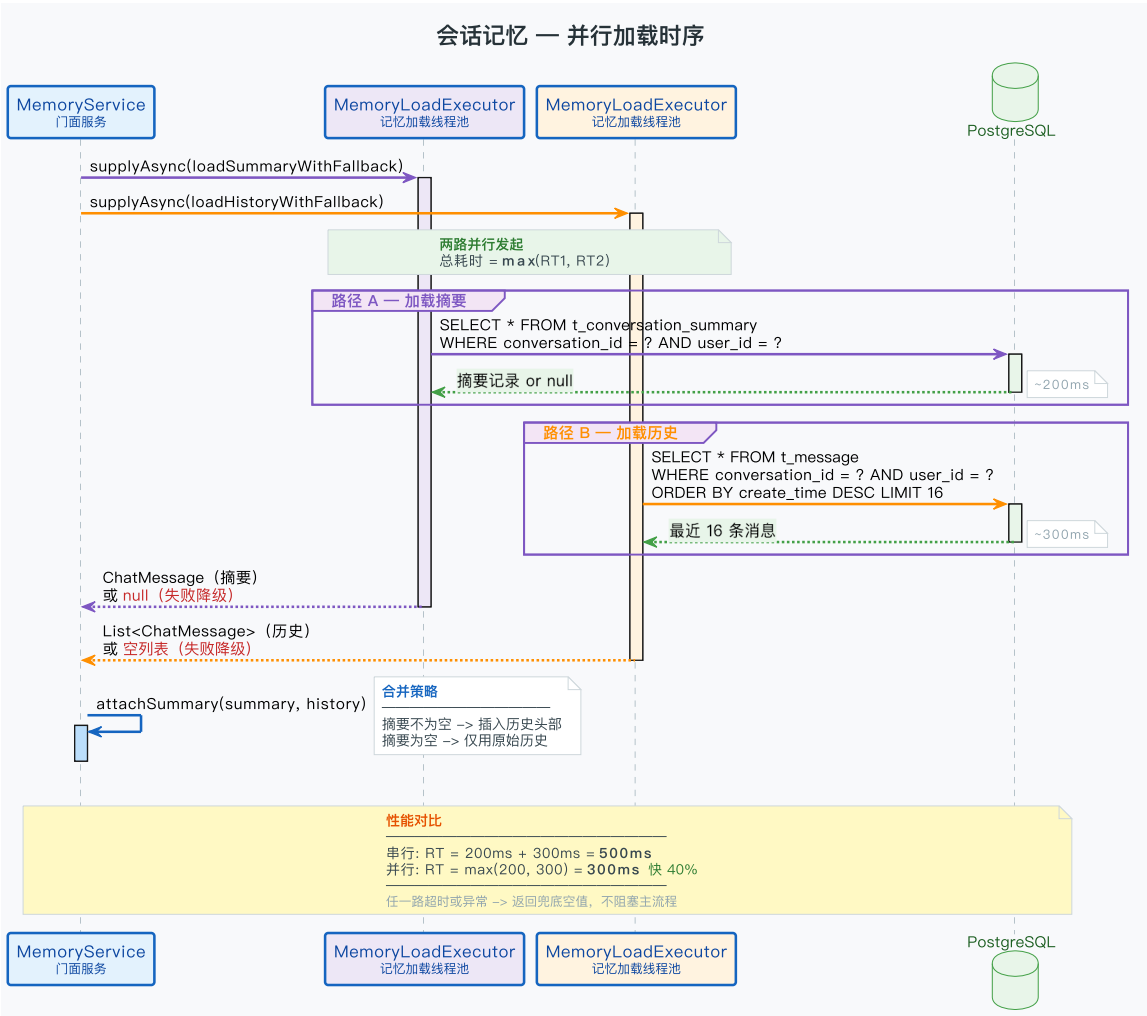
先看代码：

```
@Override
public List<ChatMessage> load(String conversationId, String userId) {
    long startTime = System.currentTimeMillis();
    try {
        // 并行加载摘要和历史记录
        CompletableFuture<ChatMessage> summaryFuture = CompletableFuture.supplyAsync(
            () -> loadSummaryWithFallback(conversationId, userId), memoryLoadExecutor
        );
        CompletableFuture<List<ChatMessage>> historyFuture = CompletableFuture.supplyAsync(
            () -> loadHistoryWithFallback(conversationId, userId), memoryLoadExecutor
        );

        // 等待所有任务完成后合并结果
        return CompletableFuture.allOf(summaryFuture, historyFuture)
            .thenApply(v -> {
                ChatMessage summary = summaryFuture.join();
                List<ChatMessage> history = historyFuture.join();
                log.debug("加载对话记忆 - conversationId: {}, userId: {}, 摘要: {}, 历史消息数: {}",
                    conversationId, userId, summary != null, history.size(), System.curren
                return attachSummary(summary, history);
            })
            .join();
    } catch (Exception e) {
        log.error("加载对话记忆失败 - conversationId: {}, userId: {}", conversationId, userId, e);
        return List.of();
    }
}
```

加载记忆需要两样东西：摘要（早期对话的浓缩版）和历史（最近 N 轮的原文）。这两样东西各需要查一次数据库，如果串行执行就是两次 RT（Round Trip），并行执行只需要 max(RT1, RT2)。

用时序图看更直观：



打个比方，这就像点外卖时你同时下了两个订单——一个奶茶、一个炒饭。它们各自在不同的厨房做，你等的时间取决于做得最慢的那个，而不是两个加起来。

2. 降级策略

并行拉取还有一个好处：两路独立隔离，一路出错不拖累另一路。
方法名里的 WithFallback 已经暗示了降级逻辑：

路径	正常返回	异常降级	降级后果
摘要路径	ChatMessage（摘要内容）	返回 null	历史正常返回，只是缺少早期对话的浓缩信息
历史路径	List<ChatMessage>（最近 N 轮）	返回空列表	当前请求继续处理，只是没有上下文

两路的降级互不影响。即使摘要表被误删了，历史照常加载；即使消息表出了问题，摘要该有的还有，当前用户消息照样能正常处理。这种隔离性在生产环境很重要——不会因为一个边缘功能的异常导致整个问答链路挂掉。

3. 滑动窗口的实现

历史不是全部加载的，而是只取最近 N 轮。这就是滑动窗口策略的工程实现。

3.1 查询策略：倒序 + LIMIT

```
@Override
public List<ChatMessage> loadHistory(String conversationId, String userId) {
    int maxMessages = resolveMaxHistoryMessages(); // historyKeepTurns * 2
    List<ConversationMessageVO> dbMessages = conversationMessageService.listMessages(
        conversationId, userId, maxMessages, ConversationMessageOrder.DESC
    );
    // ...过滤并转换为 ChatMessage
    return normalizeHistory(result);
}
```

```
private int resolveMaxHistoryMessages() {  
    int maxTurns = memoryProperties.getHistoryKeepTurns();  
    return maxTurns * 2; // 每轮 = 1 条 user + 1 条 assistant  
}
```

historyKeepTurns 默认值是 8，意味着保留最近 8 轮对话，也就是 16 条消息（每轮包含 1 条 user + 1 条 assistant）。

查询方式是 ORDER BY create_time DESC LIMIT 16——先倒序取最近的 16 条，然后在内存中反转恢复时间顺序。为什么不直接正序查？因为正序查你不知道应该从哪条开始（OFFSET 需要先算出总数），而倒序 + LIMIT 是一次查询直达目标，SQL 执行效率更高。

3.2 normalizeHistory: 确保以 USER 开头

拿到原始消息后，还有一步关键处理——normalizeHistory:

```
private List<ChatMessage> normalizeHistory(List<ChatMessage> messages) {  
    // 剥离开头的 ASSISTANT 消息，确保历史以 USER 消息开头  
    int start = 0;  
    while (start < cleaned.size() && cleaned.get(start).getRole() == ChatMessage.Role.ASSISTANT) {  
        start++;  
    }  
    return cleaned.subList(start, cleaned.size());  
}
```

为什么要确保历史切片以 USER 消息开头？大模型 API 要求 messages 数组中 user 和 assistant 交替出现。如果滑动窗口恰好从一条 assistant 消息切开了——比如第 8 轮的 assistant 回复刚好是第 17 条消息，窗口保留了它但没保留它前面的 user 消息——那历史就变成了 [assistant, user, assistant, ...]，以 assistant 开头。这会让大模型困惑：怎么第一条就是我自己的回复？前面那个人问了什么？normalizeHistory 就是干这个事的：如果切片开头是 assistant 消息，跳过它，直到碰到第一条 user 消息。宁可少一轮也不能格式乱。回到前面的 OA 系统场景，假设员工已经聊了 10 轮，滑动窗口是 8 轮。加载出来的历史大致是这样：

```
第 3 轮 user:    "加班审批通过后怎么看记录？"  
第 3 轮 assistant: "您可以在 OA 系统的..."  
第 4 轮 user:    "那调休假怎么申请？"  
第 4 轮 assistant: "调休假申请的流程是..."  
...  
第 10 轮 user:   "如果它超过三天没审批怎么办？"  
第 10 轮 assistant: （还没回复，这是当前轮）
```

第 1、2 轮的对话被窗口滑走了，不在列表里。如果那两轮里有重要信息（比如员工提到过自己是外包身份，审批流程和正式员工不同），靠滑动窗口是找不回来的——这就是第 3 篇要解决的摘要压缩问题。

4. 合并摘要和历史

两路并行加载完成后，attachSummary 把它们合在一起：

```
private List<ChatMessage> attachSummary(ChatMessage summary, List<ChatMessage> messages) {  
    if (CollUtil.isEmpty(messages)) {  
        return List.of();  
    }  
    if (summary == null) {  
        return messages;  
    }  
    List<ChatMessage> result = new ArrayList<>();  
    result.add(summaryService.decorateIfNeeded(summary)); // 摘要作为 SYSTEM 消息放在最前面  
    result.addAll(messages);  
    return result;  
}
```

逻辑很简单，但有几个边界条件的处理值得注意：

历史为空（新会话，还没聊过）→ 直接返回空列表，不管有没有摘要。因为摘要是对历史对话的浓缩，连历史都没有，摘要也没有意义。

摘要为 null（没触发过压缩，或者加载失败降级了）→ 原样返回历史列表。

两者都有 → 摘要作为 SYSTEM 消息插到列表头部，后面跟着最近 N 轮原文。

合并后的列表结构长这样：

```
[0] SYSTEM:  "对话摘要：用户之前询问了 OA 系统的加班申请流程，以及审批后的记录查看方式..."
[1] USER:    "那调休假怎么申请？"
[2] ASSISTANT: "调休假申请的流程是..."
[3] USER:    "调休假和年假有什么区别？"
[4] ASSISTANT: "主要区别在于..."
...
```

摘要放在头部是有讲究的——它给大模型提供了一段长期上下文背景，就像会议纪要的“前情提要”，让模型在读后面的原文对话之前，先对之前聊过什么有个大致印象。

loadAndAppend：时序设计

Pipeline 里调用的不是 load，也不是 append，而是一个叫 loadAndAppend 的便捷方法。这个方法的时序设计值得单独拿出来说说。

1. 为什么先加载再追加

```
default List<ChatMessage> loadAndAppend(String conversationId, String userId, ChatMessage message) {
    List<ChatMessage> history = load(conversationId, userId);
    append(conversationId, userId, message);
    return history;
}
```

调用处在 StreamChatPipeline 里：

```
private void loadMemory(StreamChatContext ctx) {
    List<ChatMessage> history = memoryService.loadAndAppend(
        ctx.getConversationId(), ctx.getUserId(), ChatMessage.user(ctx.getQuestion())
    );
    ctx.setHistory(history);
}
```

注意看——传进去的 message 是当前用户发的消息（ChatMessage.user(ctx.getQuestion())），但 loadAndAppend 返回的 history 是追加之前的历史快照，不包含当前这条消息。

为什么这样设计？

2. 为什么不把当前消息包含在历史里

想象一下，如果 loadAndAppend 先 append 再 load，返回的历史就包含了当前用户消息。然后到了阶段 8 组装 Prompt 时，Pipeline 还会把当前用户的问题作为最后一条 user 消息拼进去——结果就是用户的问题出现了两次。

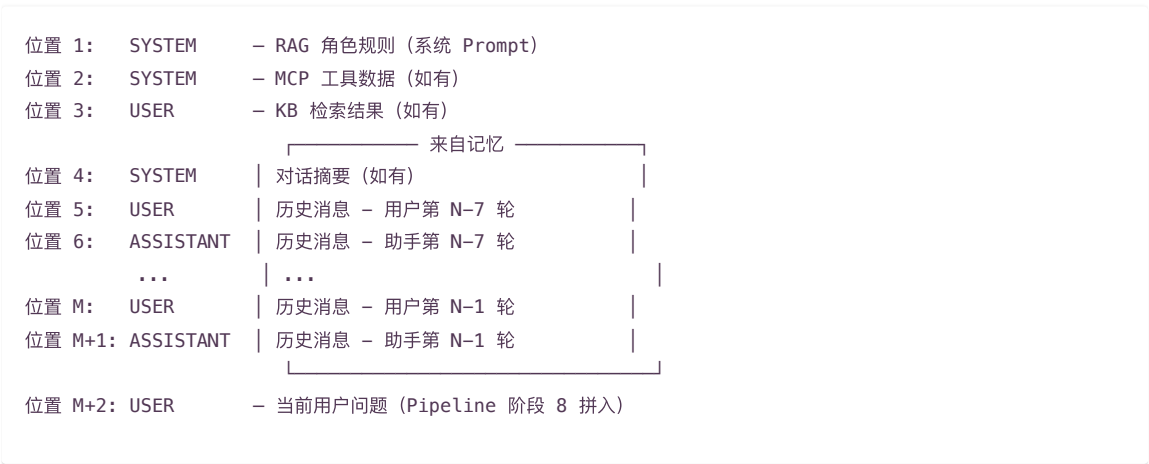
```
# 错误的情况：历史里已经包含当前消息
[...历史消息...]
USER: "如果它超过三天没审批怎么办？"  ← 历史里的
USER: "如果它超过三天没审批怎么办？"  ← Pipeline 阶段 8 拼入的（重复了！）
```

先 load 再 append，返回的历史只到上一轮为止，当前消息由 Pipeline 最后组装时单独拼入，各管各的，不会重复。

这也是一种关注点分离：**记忆服务负责管理历史，Pipeline 负责管理当前轮次**。记忆服务不需要知道当前消息在 Prompt 里的位置，Pipeline 也不需要操心历史消息是怎么存的。

记忆注入 Prompt 的完整顺序

记忆加载完成后，最终会在阶段 8 和其他上下文一起拼成发给大模型的完整消息列表。来看看这个列表的完整结构：



这个顺序不是随便排的，每个位置都有它的道理：

位置	角色	内容	设计考量
最前	SYSTEM	RAG 角色规则	优先级最高的行为约束，大模型会把最前面的 SYSTEM 消息当作最高优先级指令
次前	SYSTEM	对话摘要（如有）	长期上下文的前情提要，模型先看到浓缩背景再看原文对话，理解更连贯
中间	USER / ASSISTANT 交替	最近 N 轮原文历史	保持对话自然流转，模型读这段就像在翻聊天记录，感受话题的演进
最后	USER	当前用户问题	紧邻生成位置，模型天然对最后一条 user 消息分配最高注意力，将其作为当前要回答的问题

用 OA 系统的场景具体化一下。员工问到第 10 轮“如果它超过三天没审批怎么办”时，发给大模型的消息数组大致是这样的：

```
[1] SYSTEM: "你是企业知识库助手，只能基于检索到的内容回答用户问题..."
[2] USER: "【参考资料】[1] OA 加班申请超过 3 个工作日未审批可发起催办..."
[3] SYSTEM: "对话摘要：用户之前询问了 OA 系统的加班申请提交方式..."
[4] USER: "那调休假怎么申请？"
[5] ASSISTANT: "调休假申请的流程是..."
... (中间省略几轮)
[M] USER: "提交之后审批流程是怎样的？"
[M+1] ASSISTANT: "审批流程分为三级..."
[M+2] USER: "如果它超过三天没审批怎么办？"
```

模型读到最后一日消息，结合前面的对话历史知道“它”指的是加班申请，结合检索到的参考资料知道超过 3 个工作日可以发起催办，然后生成准确的回答。每一层上下文都在发挥作用。

配置项与调优

记忆系统的行为由 MemoryProperties 配置类控制。

1. 核心配置项一览

```
@ConfigurationProperties(prefix = "rag.memory")
@ValidMemoryConfig // 交叉校验: summaryStartTurns > historyKeepTurns
public class MemoryProperties {

    /** 保留原文的最近轮数 (user+assistant 为一轮)，默认 8，范围 [1,100] */
```


5. **时序设计**: loadAndAppend 先加载再追加, 返回的历史不含当前消息。记忆服务管历史, Pipeline 管当前轮次, 关注点分离避免消息重复。

滑动窗口解决了保留最近 N 轮的问题, 但更早的对话就直接丢掉了吗? 如果那个员工在第 2 轮提到过自己是外包身份、审批流程和正式员工不同, 到了第 12 轮系统就忘了这个关键约束?

下一篇聊摘要压缩策略——怎么把早期对话浓缩成一段摘要, 在有限的 Token 预算内保住关键信息, 同时不让压缩过程拖慢主流程。

